Advanced Generative AI



Introduction to Large Language Models and LangChain



Quick Recap



- Explain the role of the attention mechanism in the Transformer architecture. How does self-attention help in processing input sequences?
- Describe the main differences between the encoder and decoder in the Transformer architecture?

Engage and Think



Imagine you are a software developer tasked with building a customer support chatbot for an e-commerce platform. The chatbot needs to handle a wide range of queries, from product information and order tracking to troubleshooting common issues. You want the chatbot to provide accurate, context-aware responses and continuously improve its performance based on user interactions. You decide to leverage Large Language Models (LLMs) and the LangChain framework to create a robust, intelligent chatbot that can meet these requirements.

How can integrating Large Language Models with the LangChain framework help you build a more efficient and intelligent customer support chatbot for an e-commerce platform.

Learning Objectives

By the end of this lesson, you will be able to:

- Develop an understanding of the core components and architecture of Large Language Models (LLMs)
- Analyze the LLM in action and its training process, encompassing tokenization, embedding, neural network training, and fine-tuning
- Identify the functioning of LLMs, focusing on how they generate human-like text and respond to prompts
- Explore the fundamentals of LangChain to understand its features and functionality



Language Models

Language Models

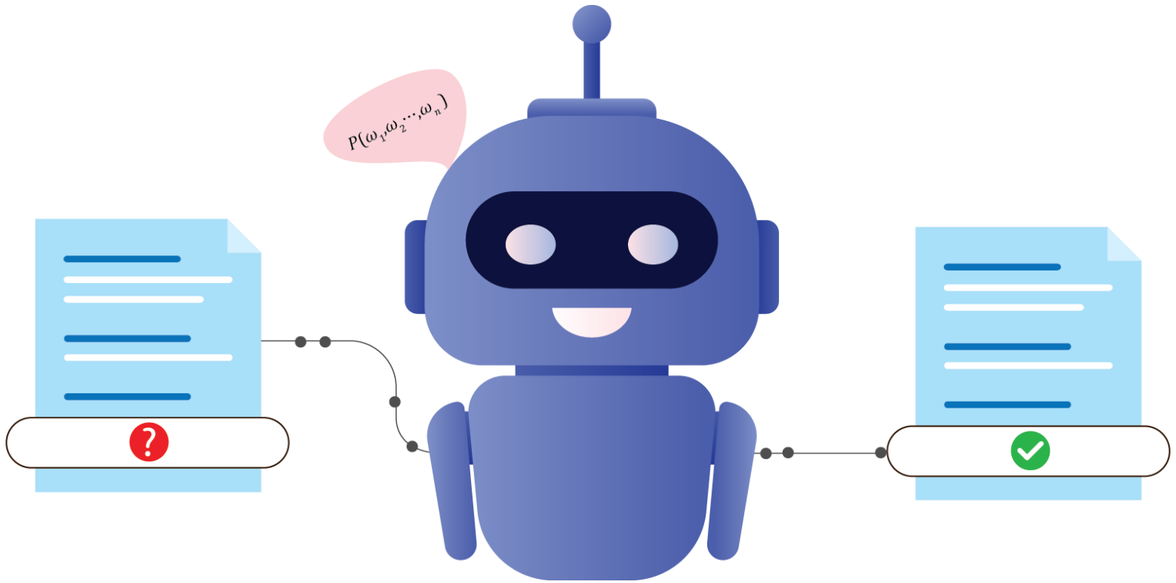
A language model is a probabilistic machine learning entity.

It resembles a complex function, designed to predict the probability of word sequences within a specific language corpus.

It is represented as: P(**Any sentence here**)

Language Models: Equation

Language models operate by assigning probabilities to sequences of words.



Mathematically, it looks like this:

$$P(\omega_1, \omega_2 \cdots, \omega_n) = P(\omega_1) \cdot P(\omega_2|\omega_1) \cdot P(\omega_3|\omega_1, \omega_2) \cdot \ldots \cdot P(\omega_n|\omega_1, \omega_2, \ldots, \omega_{n-1})$$

Language Models: Example

Consider the sentence: This is a new technology.

The language model calculates the probability of the sentence as:

P**(This is a new technology)**

$P(\text{This is a new technology}) = P(\text{This})\ P(\text{is}|\text{This})\ P(\text{a}|\text{This is})\ P(\text{new}|\text{This is a})\ P(\text{technology}|\text{This is a new})$

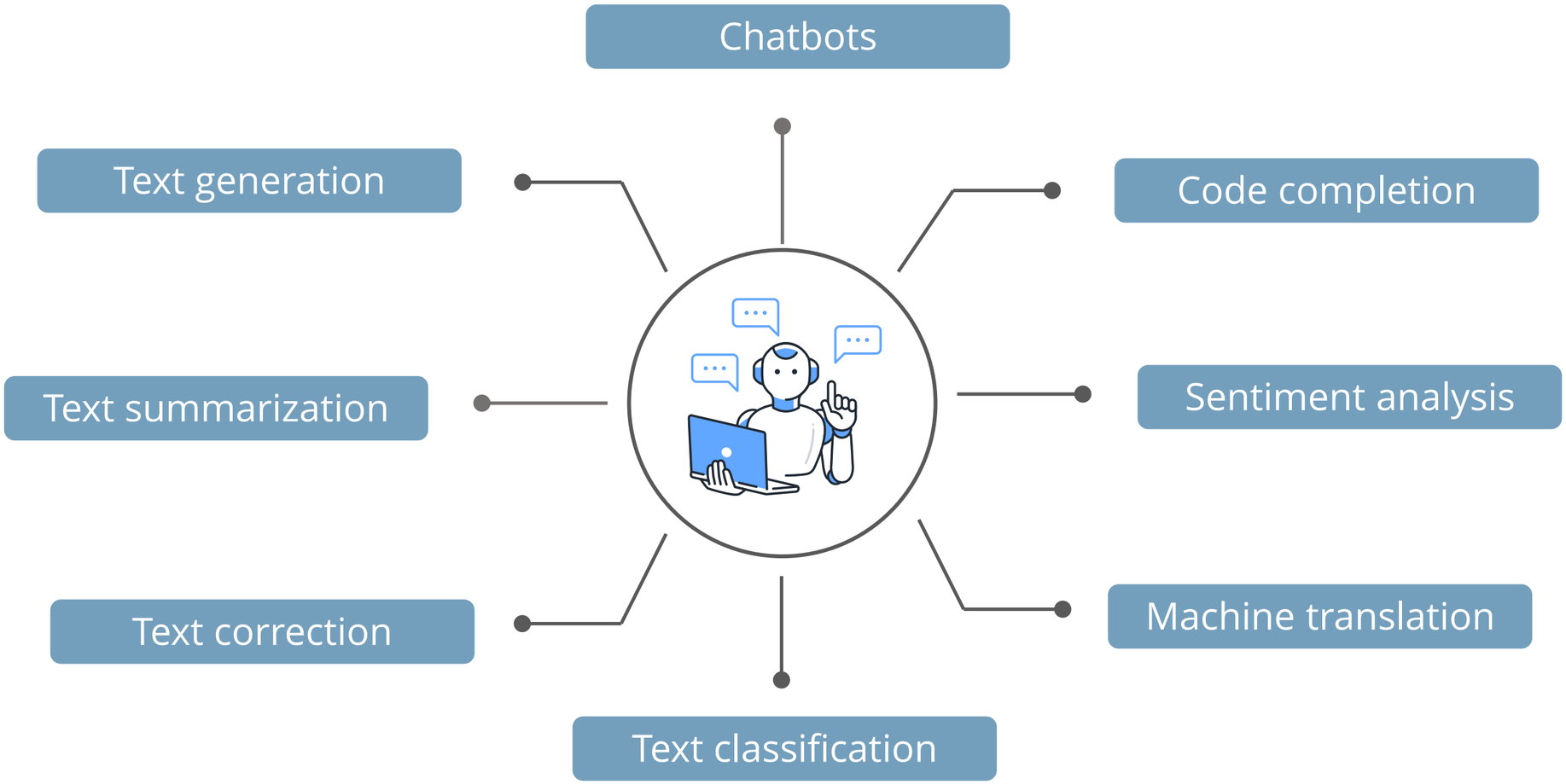
Language Models: Calculation

To illustrate, let's calculate the probability of two different sentences:

1. P(This is a fluffy dog.)
2. P(This are a purple flying deer.)

Solution: Sentence 1 gets a high probability, leveraging common context, and in sentence 2, rare and challenging words result in a lower probability.

Applications of Language Models



Demo: Text Generation



Duration: 20 minutes

Imagine you are on a quest to understand the intricate art of text generation, where a computer learns the patterns of a given writing style and crafts its sentences.

Today's session will explore a Python script designed for educational purposes. This script employs the Natural Language Toolkit (NLTK) and the Brown corpus to demonstrate text generation through a Markov chain model using trigrams.

Note

Please download the solution document from the Reference Material Section and follow the Jupyter Notebook for step-by-step execution.

DEMONSTRATION

Quick Check



Which of the following is not an application of language models?

A. Text generation

B. Machine translation

C. Speech recognition

D. Image processing

Large Language Models

Large Language Models

Large Language Models (LLMs) are state-of-the-art AI models designed to comprehend and generate human language.

Large	Refers to the significant size and complexity of these models, which contains hundreds of millions or even billions of parameters
Language	Denotes their primary function, which is to understand and generate human language
Model	Describes them as mathematical representations that capture the patterns and structure of language data

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

This process involves breaking down text into smaller units called tokens, which can be words, phrases, or even individual characters.

Transfer learning

Encoder and decoder

Scaling

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

This embedding component maps tokens to a high-dimensional vector space, representing each token with a unique vector.

Transfer learning

Encoder and decoder

Scaling

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

The attention mechanism lets the model concentrate on specific parts of the input text when generating output.

Transfer learning

Encoder and decoder

Scaling

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

Pretraining in LLMs involves extensively training them on large volumes of text data to capture and learn the intricate patterns and structures inherent in human language.

Transfer learning

Encoder and decoder

Scaling

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

This component allows the model to adapt to new tasks by fine-tuning the pre-trained model on a smaller dataset.

Transfer learning

Encoder and decoder

Scaling

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

This employs the Transformer framework in a large language model architecture, comprising two main parts: an encoder and a decoder.

Transfer learning

Encoder and decoder



Scaling

Components of LLMs

Tokenization

Embedding

Attention

Pretraining

This necessitates significant computational resources for training and upkeep, making scaling a challenging but essential part of its architecture.

Transfer learning

Encoder and decoder

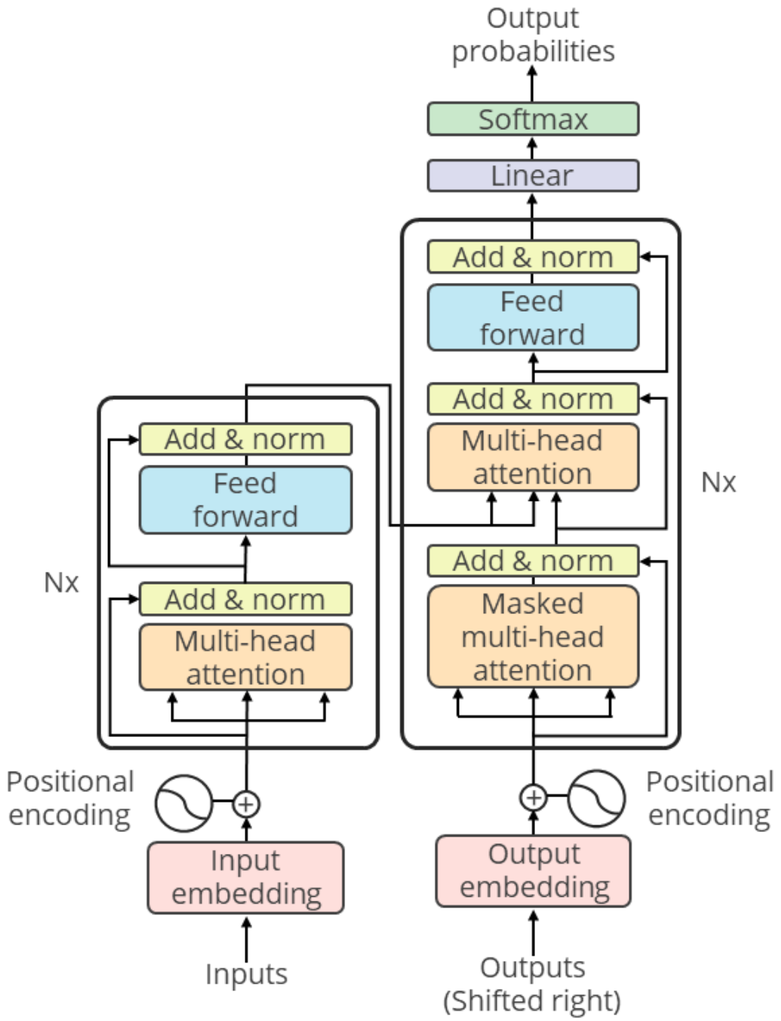
Scaling



LLM Architecture

Components of LLM architecture

- Input embeddings
- Positional encoding
- Encoder
  - Attention mechanism
  - Feed-forward neural network
- Decoder
- Multi-headed attention
- Layer normalization
- Output



LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- The input text is converted into a format the model understands, called embeddings.
- These embeddings are high-dimensional vectors that capture the semantic properties of words.

LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- Since the model does not inherently understand the order of words in a sentence, positional encodings are added.
- It is added to the input embeddings to provide contextual information about the position of words within the sentence

LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- **Encoder:** The encoder processes the input embeddings (augmented with positional encodings) through multiple layers.
- **Attention mechanism:** Each layer applies self-attention mechanisms that allow the model to weigh the importance of different words relative to each other.
- **Feed forward:** It is followed by feed-forward neural networks for further transformation.

LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- In models that include a decoder (like the Transformer used in machine translation), the decoder generates output text by processing the encoder's output along with its previous outputs.
- The decoder helps the machine predict what word comes next based on what it has understood so far.

LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- This mechanism allows the model to focus on different positions of the input sequence simultaneously.
- It splits the attention mechanism into multiple 'heads', each capable of capturing different types of dependencies in the input data.

LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- This is a technique to standardize the inputs to each layer within the network,
- It stabilizes the learning process and improves the speed of training.

LLM Operations

These represent the functions of components within an architecture.

- Input embeddings
- Positional encoding
- Encoder
- Decoder
- Multi-headed attention
- Layer normalization
- Output

- Finally, the model produces its own understanding or generates new sentences.
- The output depends on what the model is designed to do.
- For example, if it's predicting the next word in a sentence, it gives a probability for each word.

LLM Training Steps

The steps in the training process of a language model are:

- Corpus preparation
- Tokenization



- Neural network training
- Embedding generation

Types of LLMs

Below are the various pretrained LLMs available in the market:



GPT 3.5
and GPT 4



PaLM



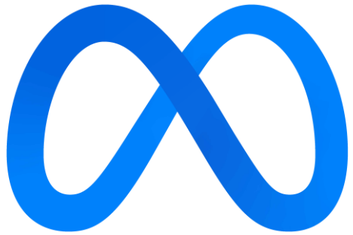
Claude



Cohere



Falcon



LLaMA

Quick Check



When considering the architecture of Large Language Models (LLMs), which of the following components is responsible for generating human-like text and responding to prompts?

A. Tokenization

B. Embedding

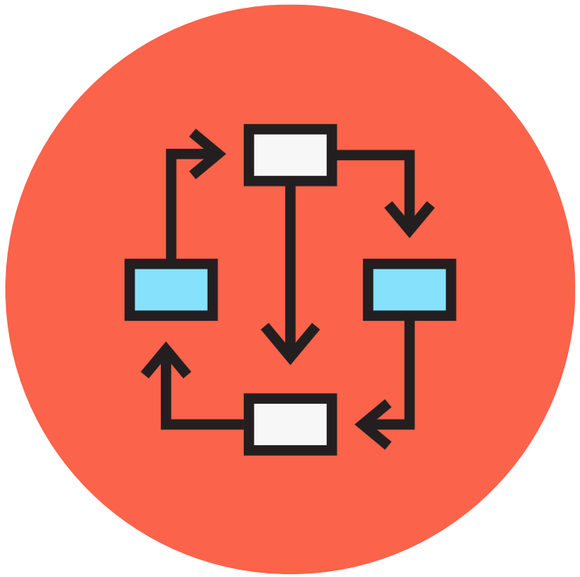
C. Neural network training

D. Fine-tuning

Introduction to LangChain

Background of LangChain

The purpose of LangChain is to simplify the process of creating generative AI applications.



- It was launched as an open-source project in 2022.
- It is already packed with incredible features for building tools around the core of language models.
- It is part of a rich ecosystem of tools that integrate with the framework and build on top of it.

Introduction to LangChain

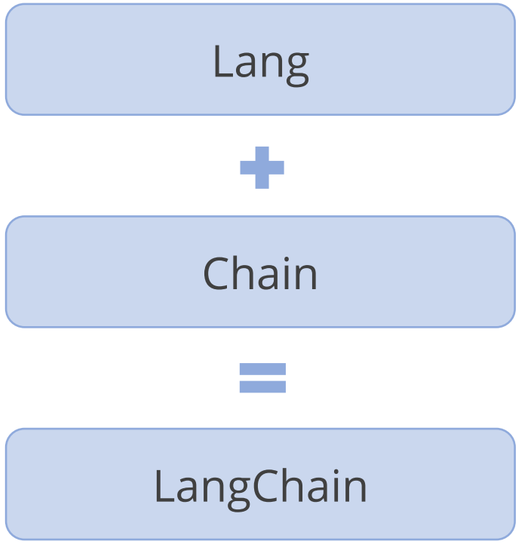
LangChain is an open-source framework for developing applications powered by language models.



It simplifies linking language models to data, environments, and applications.

Decoding the Name

The name **LangChain** is a merged word of **Lang** and **Chain**.



- **Lang:** It is the short form of the term **language**.
- **Chain:** It emphasizes the connection of various elements to create advanced applications around LLMs.

LangChain: Key Concepts



Framework Overview: LangChain is designed to **chain** the different components together to create advanced use cases around LLMs.



Applications: It has applications in various fields, including machine learning, translation services, content creation, and more.



Features: The key features include immutability, peer-to-peer networks, and linguistic asset ownership, which could revolutionize how linguistic data is handled in the digital age.

How LangChain Works?

LangChain simplifies the process of building these applications by providing a set of tools and abstractions.

The provided set of tools and abstractions makes it easier to:

- Connect language models to other data sources
- Interact with their environment
- Build complex applications

LangChain: Working Analogy



LangChain works by chaining a series of components called links to create a workflow.



Each link in the chain performs a specific task.



The links in a chain are connected in a sequence, and the output of one link is passed as the input to the next link.

Compatibility: Programming Languages

LangChain is compatible with the Python and JavaScript programming languages.

It provides support for Python and JavaScript, allowing developers to:

- Leverage the power of LangChain in web-based and Node.js applications
- Take advantage of the robust type-safety features

Note

LangChain is currently considering the development of versions in Java and C++.

Quick Check



How does LangChain operate to accomplish tasks?

A. Through random execution of links

B. By connecting links in a sequence

C. Using parallel processing for all links

D. Independent execution of each link

Architecture of LangChain

LangChain: Framework

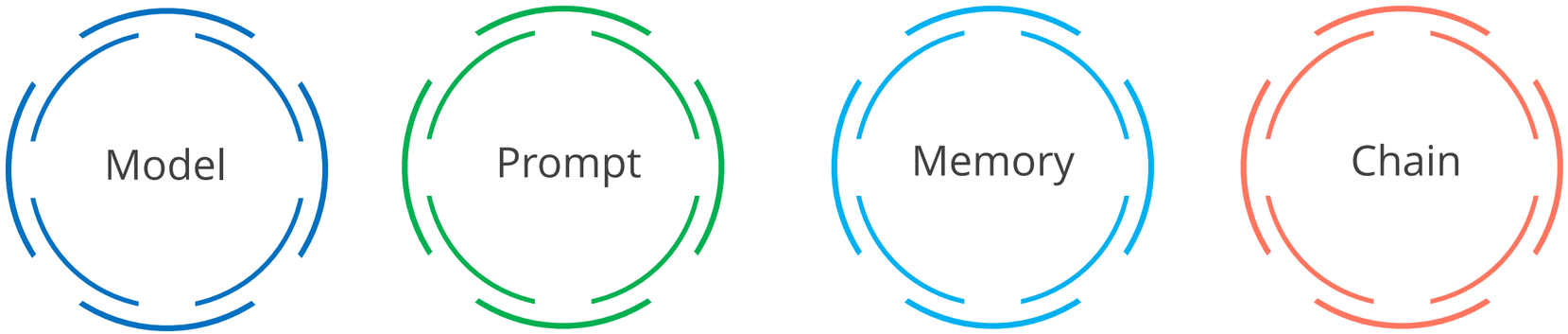
The LangChain framework enables applications that are:

Context-aware: Connect a language model to sources of context (prompt instructions, few-shot examples, content to ground its response in, and so on).

Reason-based: Rely on a language model to reason (about how to answer based on the provided context, what actions to take, and so on).

LangChain: Components

LangChain has four key components:

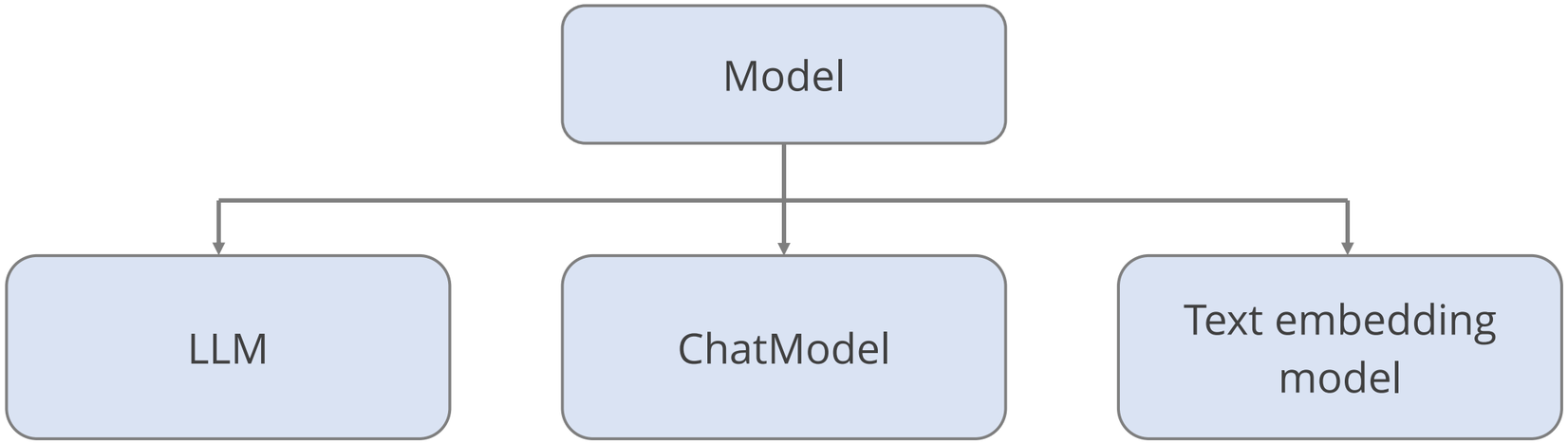


Note

These constitute components within LangChain, distinct from the components of LangChain's architecture.

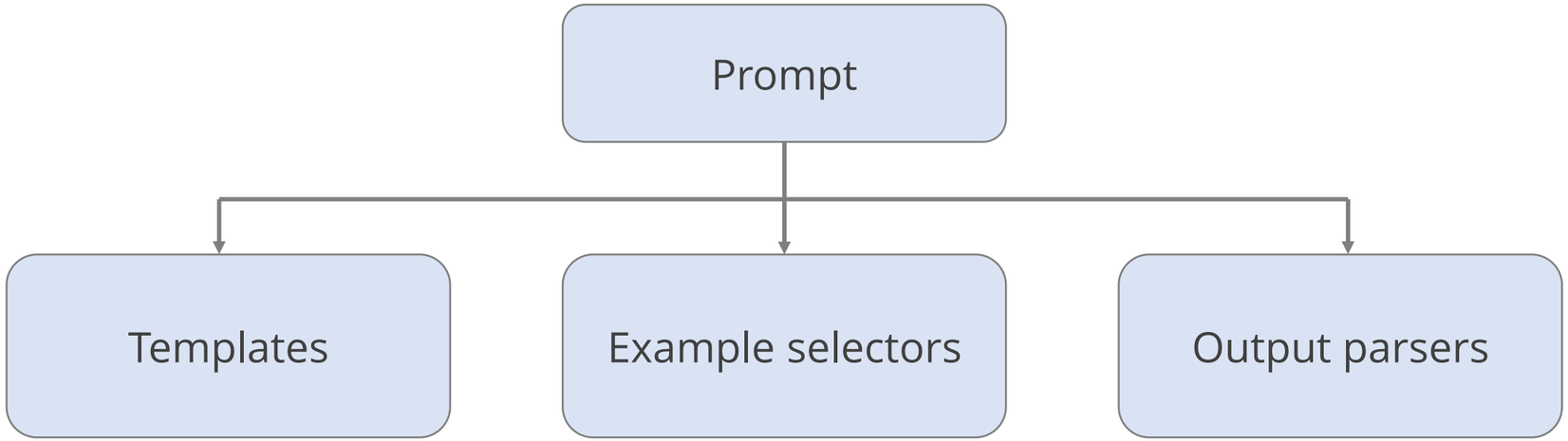
Model

Model is the core of LangChain, consisting of three primary types:



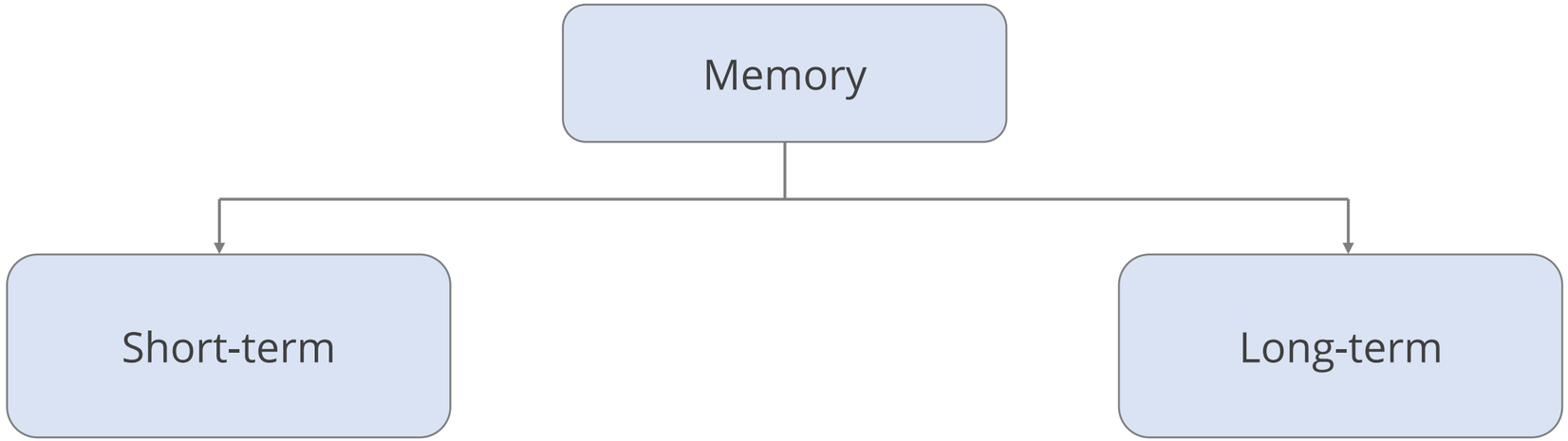
Prompt

Prompt is the entry point for interacting with LLMs, which comprises of:



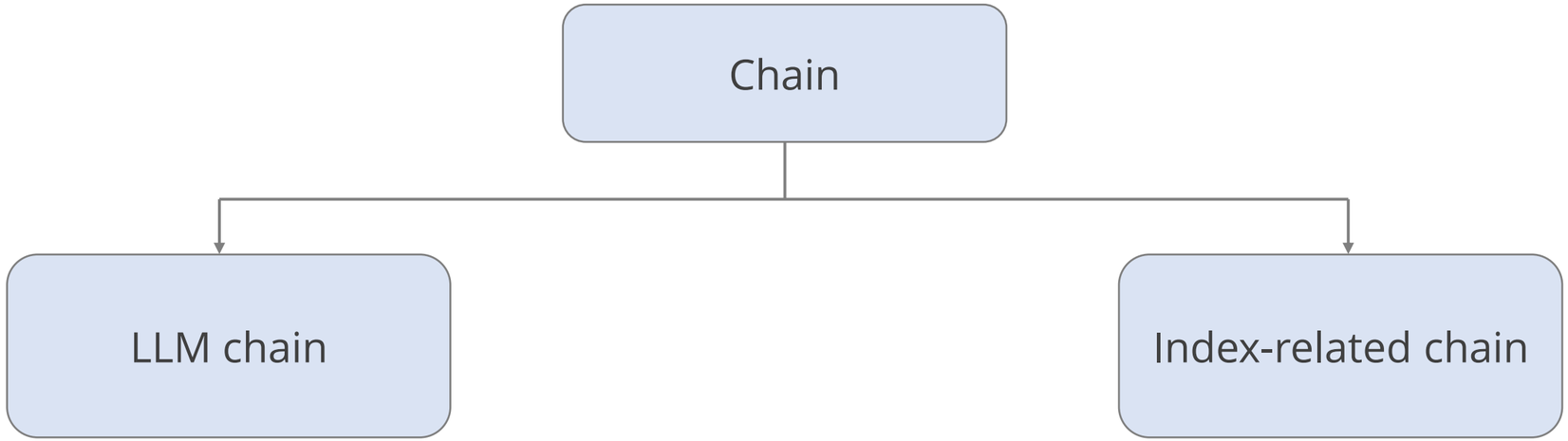
Memory

Memory plays a crucial role in creating a seamless and interactive experience, and it is divided into:



Chain

Chain brings together various elements to create meaningful responses and has two common types:

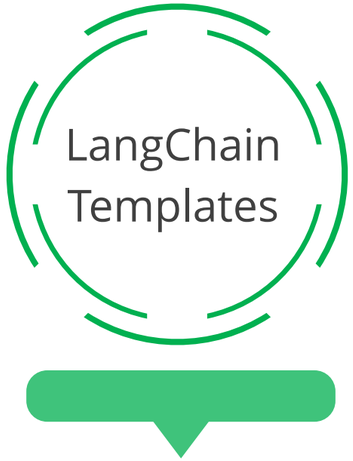


Architecture of LangChain Framework

The components of LangChain architecture consists of:



A library of Python and JavaScript with interfaces and integrations



A collection of easily deployable reference architectures



A library for deploying LangChain chains as a REST API



A developer platform that allows debugging, testing, evaluation, and monitoring of chains

Quick Check



What is the purpose of LangServe in the LangChain architecture?

A. Debugging and testing chains

B. Creating reference architectures

C. Deploying LangChain chains as a REST API

D. Developing interfaces and integrations

Working with LangChain

System Requirements for LangChain



Operating System

LangChain is compatible with modern operating systems such as Windows, Mac OS, or Linux.

Storage and memory

Systems must have sufficient storage space and memory to handle the installation and operations.

Installation of LangChain

These commands will install the minimum requirements of LangChain.

To install LangChain, you can use the following commands:

- Pip:

```bash
pip install langchain
```

- Conda:

```bash
conda install langchain -c conda-forge
```

To integrate it with various model providers, data stores, and more, you will need to install the dependencies for specific integrations separately.

Note

The libraries and dependencies are already preinstalled in the Simplilearn lab, so, refrain from using the **pip install** command.

Building Applications with LangChain

Once LangChain is installed, you can start building your language model application.

The three most common and important are:

- **LLMs:** LangChain provides support for large language models, which can be configured and used for various applications.
- **Chains:** Chains go beyond a single LLM call and involve sequences of calls, allowing for more complex applications.
- **Prompts:** Prompts are input to LLMs and can be structured in different ways to get different results.

Understanding these concepts will set you up well for being able to use and customize LangChain applications.

Building Applications with LangChain: Example

LangChain enables the development of various generative AI applications, which include but not limited to:

- Chatbots
- Text summarization
- Generative question-answering
- Code understanding and writing
- Data analysis and extraction
- Web scraping

LangChain can build chatbots or personal assistants that leverage large language models to interact with users and provide intelligent responses.

Building Applications with LangChain: Example

LangChain enables the development of various generative AI applications, which include but not limited to:

- Chatbots
- Text summarization
- Generative question-answering
- Code understanding and writing
- Data analysis and extraction
- Web scraping

LangChain allows efficient development of text summarization applications, condensing large bodies of text into shorter versions.

Building Applications with LangChain: Example

LangChain enables the development of various generative AI applications, which include but not limited to:

- Chatbots
- Text summarization
- Generative question-answering
- Code understanding and writing
- Data analysis and extraction
- Web scraping

LangChain can be utilized to create applications for generating answers to questions based on the input provided, enhancing the capabilities of question-answering systems.

Building Applications with LangChain: Example

LangChain enables the development of various generative AI applications, which include but not limited to:

- Chatbots
- Text summarization
- Generative question-answering
- Code understanding and writing
- Data analysis and extraction
- Web scraping

LangChain can be employed to develop applications for understanding and writing code, leveraging the power of language models to assist developers in coding tasks.

Building Applications with LangChain: Example

LangChain enables the development of various generative AI applications, which include but not limited to:

- Chatbots
- Text summarization
- Generative question-answering
- Code understanding and writing
- Data analysis and extraction
- Web scraping

LangChain can be used for analyzing structured data, extracting information, and interacting with APIs, providing powerful capabilities for data-driven applications.

Building Applications with LangChain: Example

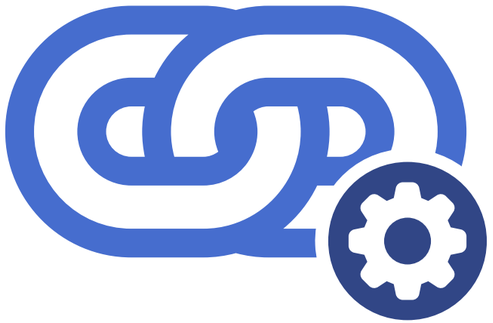
LangChain enables the development of various generative AI applications, which include but not limited to:

- Chatbots
- Text summarization
- Generative question-answering
- Code understanding and writing
- Data analysis and extraction
- Web scraping

LangChain can be utilized for web scraping applications, enabling the extraction of data from websites for various purposes.

Customizing Language Model

LangChain is a powerful platform that simplifies model customization for developers.



- LangChain simplifies model customization, helping developers tailor pre-trained models for specific tasks.
- The LLM class eases interaction with providers such as OpenAI and Hugging Face, streamlining language model engagement, component interlinking, and asset incorporation.

Note

Standard autoencoders' limitations have led to the development of Variational Autoencoders (VAEs). VAEs overcome these challenges and create a more versatile and effective generative model.

Quick Check



Which of the following is a feature supported by LangChain to facilitate language model engagement, component interlinking, and asset incorporation?

A. LLMs

B. Chains

C. Prompts

D. Templates

A Comprehensive Overview of LangChain

Features of LangChain



Composable tools and integrations: LangChain provides composable tools and integrations for working with language models.



Off-the-shelf chains: LangChain offers built-in assemblages of components for accomplishing higher-level tasks.



Immutability and security: LangChain enhances security, transparency, and decentralization in language data management, leveraging peer-to-peer networks for efficient and secure data exchange.



Linguistic asset ownership: LangChain introduces the concept of linguistic asset ownership, revolutionizing how linguistic data is handled in the digital age.

Benefits of LangChain



Simplified development: LangChain enables quick and easy development of generative AI applications.



Flexibility and customization: LangChain offers flexibility and customization options.



Seamless integration: LangChain provides an application programming interface (API) facilitating seamless integration.

Data Privacy and Security

- Comprehensive security program
- Security awareness and training
- Data anonymization
- Confidential computing
- Controlled data logging
- Immutability and decentralization

Best Practices

- API reference and documentation
- Security considerations
- Prompt engineering
- Chaining components
- Tool selection and integration

Demo: Building a Text Generation Pipeline with LangChain and Hugging Face's Flan T5 XXL Model



Duration: 20 minutes

Overview:

In this demo, you will learn how to create a Langchain HuggingFacePipeline for efficient text generation and dive into the creation of a Langchain chain to craft context-aware responses using a custom template.

Note

Please download the solution document from the Reference Material Section and follow the Jupyter Notebook for step-by-step execution.

DEMONSTRATION

Quick Check



Which feature of LangChain contributes to security, transparency, and decentralization in language data management?

A. Off-the-Shelf Chains

B. Linguistic Asset Ownership

C. Context-Aware Applications

D. Rich Ecosystem and Integrations

Key Takeaways

- Large Language Models are trained on large datasets, and they can generate human-like text, images, and many more.
- Pretrained LLMs available in the market can be utilized for powerful generative AI solutions
- LangChain serves as an open-source framework designed for the development of applications driven by language models.
- The LangChain architecture includes LangChain libraries, LangChain templates, LangServe, and LangSmith as its main components



Q&A

